ASSIGNMENT-16.4

SINDHU KATAKAM

BATCH-41

2303A52481

Task 1: Hospital Management Database

Prompt:

Create a complete SQL solution for a Hospital Management System by designing a well-structured database with three tables: Doctors, Patients, and Appointments. Define primary keys and foreign keys, and use constraints like NOT NULL and UNIQUE with data types that are compatible with SQLite. Then, write SQL queries using joins to display all appointments for a specific doctor using doctor_id, retrieve the full history of a patient using patient_id, and count the total number of different patients treated by each doctor. Also, include sample INSERT statements to test the database and a few SELECT queries to verify that the data is correct. Ensure that the final SQL code is clean, well-organized, and executes in SQLite without any errors.

Code:

```
-- SQLite
-- DROP old tables (important if you ran before)
DROP TABLE IF EXISTS Appointments;
DROP TABLE IF EXISTS Doctors;
DROP TABLE IF EXISTS Patients;

-- Create Doctors table CREATE TABLE Doctors (
    doctor_id INTEGER PRIMARY KEY AUTOINCREMENT,    first_name TEXT NOT NULL,
last_name TEXT NOT NULL,    specialty TEXT NOT NULL,
    UNIQUE (first_name, last_name)
);

-- Create Patients table CREATE TABLE Patients (    patient_id INTEGER PRIMARY KEY
AUTOINCREMENT,    first_name TEXT NOT NULL,    last_name TEXT NOT NULL,
date_of_birth TEXT NOT NULL,
    UNIQUE (first_name, last_name, date_of_birth)
);

-- Create Appointments table CREATE TABLE Appointments (    appointment_id INTEGER PRIMARY
KEY AUTOINCREMENT,    doctor_id INTEGER NOT NULL,    patient_id INTEGER NOT NULL,
appointment_date TEXT NOT NULL,
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
);

-- Insert data
INSERT INTO Doctors (first_name, last_name, specialty) VALUES
```

```sql
('John', 'Doe', 'Cardiology'),
('Jane', 'Smith', 'Neurology');

INSERT INTO Patients (first_name, last_name, date_of_birth) VALUES
('Alice', 'Johnson', '1980-05-15'),
('Bob', 'Brown', '1990-08-22');

INSERT INTO Appointments (doctor_id, patient_id, appointment_date) VALUES
(1, 1, '2024-07-01'),
(1, 2, '2024-07-02'),
(2, 1, '2024-07-03');

-- TEST OUTPUT (IMPORTANT)
SELECT * FROM Doctors;
SELECT * FROM Patients;
SELECT * FROM Appointments;

-- Required Queries
SELECT a.appointment_id, d.first_name, d.last_name, p.first_name, p.last_name, a.appointment_date
FROM Appointments a
JOIN Doctors d ON a.doctor_id = d.doctor_id
JOIN Patients p ON a.patient_id = p.patient_id
WHERE a.doctor_id = 1;

SELECT a.appointment_id, d.first_name, d.last_name, a.appointment_date FROM Appointments a
JOIN Doctors d ON a.doctor_id = d.doctor_id
WHERE a.patient_id = 1;

SELECT d.doctor_id, d.first_name, d.last_name, COUNT(DISTINCT a.patient_id) FROM Doctors d
LEFT JOIN Appointments a ON d.doctor_id = a.doctor_id
GROUP BY d.doctor_id;
CREATE TABLE test123 (id INTEGER);
SELECT name FROM sqlite_master;
```

Output:

SQL < 1 / 1 > 1 - 2 of 2

appointment_id	first_name	last_name	first_name	last_name	appointment_date
1	John	Doe	Alice	Johnson	2024-07-01
2	John	Doe	Bob	Brown	2024-07-02

SQL < 1 / 1 > 1 - 2 of 2

appointment_id	first_name	last_name	appointment_date
1	John	Doe	2024-07-01
3	Jane	Smith	2024-07-03

SQL < 1 / 1 > 1 - 2 of 2

doctor_id	first_name	last_name	COUNT(DISTINCT a.patient_id)
1	John	Doe	2
2	Jane	Smith	1

JUSTIFICATION:

The database is created using three connected tables, where the Appointments table connects the Doctors and Patients tables using foreign keys. This helps keep the data organized and avoids duplication. SQL joins are used to combine information from these tables so that we can easily get details like doctor appointments, a patient's history, and the total number of patients treated by each doctor.

Task 2: Real-Time Applica on: Online Shopping Database

Prompt:

Create a complete SQL solution for an online shopping (E-commerce) system by designing a well-organized database with four tables: Users, Products, Orders, and OrderDetails. Define primary keys and foreign keys, use suitable data types, and apply constraints like NOT NULL and UNIQUE so that it works properly in SQLite. Make sure the tables are properly connected and the database follows good design rules (up to Third Normal Form). Then, write SQL queries using joins to get all orders made by a specific user using user_id, find the product that was purchased the most based on quantity, and calculate the total revenue for a particular month. Also include sample INSERT statements to test the database and some SELECT queries to check the data. Finally, add a few simple suggestions to improve database design and make queries run faster. Ensure the entire SQL code is clear, well-structured, and runs without errors.

Code:

```
-- SQLite
-- SQLite E-commerce Database (FINAL CLEAN)

PRAGMA foreign_keys = ON;

-- RESET (important to avoid duplicate/exists errors)
DROP TABLE IF EXISTS OrderDetails;
DROP TABLE IF EXISTS Orders;
DROP TABLE IF EXISTS Products;
DROP TABLE IF EXISTS Users;

-- Create Users table CREATE TABLE Users (   user_id INTEGER PRIMARY KEY
AUTOINCREMENT,   username TEXT NOT NULL UNIQUE,   email TEXT NOT
NULL UNIQUE,     password TEXT NOT NULL,
   created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Create Products table CREATE TABLE Products (
   product_id INTEGER PRIMARY KEY AUTOINCREMENT,   product_name TEXT
NOT NULL,   price REAL NOT NULL,   stock INTEGER NOT NULL,   created_at
DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Create Orders table CREATE TABLE Orders (
   order_id INTEGER PRIMARY KEY AUTOINCREMENT,    user_id INTEGER NOT NULL,
   order_date DATETIME DEFAULT CURRENT_TIMESTAMP,   FOREIGN KEY (user_id)
REFERENCES Users(user_id)
```

```sql
);

-- Create OrderDetails table CREATE TABLE OrderDetails (
order_detail_id INTEGER PRIMARY KEY AUTOINCREMENT,
order_id INTEGER NOT NULL,     product_id INTEGER NOT NULL,
quantity INTEGER NOT NULL,     price REAL NOT NULL,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);

-- Insert data
INSERT INTO Users (username, email, password) VALUES
('john_doe', 'john@example.com', 'password123');

INSERT INTO Products (product_name, price, stock) VALUES
('Laptop', 999.99, 10),
('Mouse', 29.99, 50);

INSERT INTO Orders (user_id) VALUES (1);

INSERT INTO OrderDetails (order_id, product_id, quantity, price) VALUES
(1, 1, 1, 999.99),
(1, 2, 2, 29.99);

-- TEST OUTPUT (you MUST see data)
SELECT * FROM Users;
SELECT * FROM Products;
SELECT * FROM Orders;
SELECT * FROM OrderDetails;

-- 1. Retrieve all orders by a user
SELECT o.order_id, o.order_date, od.product_id, od.quantity, od.price FROM Orders o
JOIN OrderDetails od ON o.order_id = od.order_id
WHERE o.user_id = 1;

-- 2. Most purchased product
SELECT p.product_name, SUM(od.quantity) AS total_quantity_sold
FROM Products p
JOIN OrderDetails od ON p.product_id = od.product_id
GROUP BY p.product_id
ORDER BY total_quantity_sold DESC
LIMIT 1;

-- 3. Total revenue in a given month
SELECT SUM(od.quantity * od.price) AS total_revenue FROM Orders o
JOIN OrderDetails od ON o.order_id = od.order_id
WHERE strftime('%Y-%m', o.order_date) = '2024-01';
```

Output:

SQL ▾ < 1 / 1 > 1 - 2 of 2

```
SELECT o.order_id, o.order_date, od.product_id, od.quantity, od.price
FROM Orders o
JOIN OrderDetails od ON o.order_id = od.order_id
WHERE o.user_id = 1;
```

order_id	order_date	product_id	quantity	price
1	2026-03-18 13:18:04	1	1	999.99
1	2026-03-18 13:18:04	2	2	29.99

SQL ▾ < 1 / 1 > 1 - 1 of 1

```
SELECT p.product_name, SUM(od.quantity) AS total_quantity_sold
FROM Products p
JOIN OrderDetails od ON p.product_id = od.product_id
GROUP BY p.product_id
ORDER BY total_quantity_sold DESC
LIMIT 1;
```

product_name	total_quantity_sold
Mouse	2

SQL ▾ < 1 / 1 > 1 - 1 of 1

```
SELECT SUM(od.quantity * od.price) AS total_revenue
FROM Orders o
JOIN OrderDetails od ON o.order_id = od.order_id
WHERE strftime('%Y-%m', o.order_date) = '2024-01';
```

total_revenue
NULL

Justification:

The database is designed using multiple connected tables, where the OrderDetails table links the Orders and Products tables. This helps keep the data well-organized and avoids repeating the same information. SQL joins and aggregate functions are used to combine and analyze the data, making it easy to track user orders, find the most popular products, and calculate the total revenue for each month efficiently.

Task 3: Library Database

Prompt:

Create a complete SQL solution for a Library Management System by designing a well-organized database with three tables: Books, Members, and Loans. Define primary keys and foreign keys, use suitable data types, and apply constraints like NOT NULL and UNIQUE so that it works correctly in SQLite. Make sure the tables are properly connected and the database follows good design rules (up to Third Normal Form). Then, write SQL queries using joins to get all books that are currently issued and not yet returned, find overdue books where the loan date is more than 30 days from today, and count how many books each member has borrowed. Also, suggest a simple indexing strategy to improve performance. Include sample INSERT statements to test the database, and ensure the final SQL code is clear, well-structured, and runs without any errors.

Code:

-- SQLite
--Generate a complete SQL solution for a Library Management System by creating a normalized database schema with three tables: Books, Members, and Loans. Include primary keys, foreign keys, appropriate data types, and constraints such as NOT NULL and UNIQUE, ensuring compatibility with SQLite. Establish proper relationships between tables and maintain at least Third Normal Form (3NF). Then write SQL queries using joins to retrieve all books currently issued (not yet returned), find overdue books where the loan date is more than 30 days from the current date, and count the number of books loaned by each

member. Also suggest an indexing strategy to improve query performance. Include sample INSERT statements for testing and ensure the output is clean, well-structured, and fully executable without errors.

```sql
-- Create the Books table
CREATE TABLE Books (
    BookID INTEGER PRIMARY KEY AUTOINCREMENT,
    Title TEXT NOT NULL,
    Author TEXT NOT NULL,
    PublishedYear INTEGER NOT NULL,
    UNIQUE(Title, Author)
);
-- Create the Members table
CREATE TABLE Members (
    MemberID INTEGER PRIMARY KEY AUTOINCREMENT,
    FirstName TEXT NOT NULL,
    LastName TEXT NOT NULL,
    Email TEXT NOT NULL UNIQUE
);
-- Create the Loans table
CREATE TABLE Loans (
    LoanID INTEGER PRIMARY KEY AUTOINCREMENT,
    BookID INTEGER NOT NULL,
    MemberID INTEGER NOT NULL,
    LoanDate DATE NOT NULL,
    ReturnDate DATE,
    FOREIGN KEY (BookID) REFERENCES Books(BookID),
    FOREIGN KEY (MemberID) REFERENCES Members(MemberID)
);
-- Sample INSERT statements for testing
INSERT INTO Books (Title, Author, PublishedYear) VALUES ('The Great Gatsby', 'F. Scott Fitzgerald', 1925);
INSERT INTO Books (Title, Author, PublishedYear) VALUES ('To Kill a Mockingbird', 'Harper Lee', 1960);
INSERT INTO Books (Title, Author, PublishedYear) VALUES ('1984', 'George Orwell', 1949);
INSERT INTO Members (FirstName, LastName, Email) VALUES ('John', 'Doe',
'john.doe@example.com');
INSERT INTO Members (FirstName, LastName, Email) VALUES ('Jane', 'Smith',
'jane.smith@example.com');
INSERT INTO Loans (BookID, MemberID, LoanDate) VALUES (1, 1, '2024-05-01');
INSERT INTO Loans (BookID, MemberID, LoanDate) VALUES (2, 2, '2024-05-15');
-- Query to retrieve all books currently issued (not yet returned)
SELECT b.Title, b.Author, m.FirstName, m.LastName, l.LoanDate FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE l.ReturnDate IS NULL;
-- Query to find overdue books where the loan date is more than 30 days from the current date
SELECT b.Title, b.Author, m.FirstName, m.LastName, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE l.ReturnDate IS NULL AND julianday('now') - julianday(l.LoanDate) > 30;
-- Query to count the number of books loaned by each member
```

```sql
SELECT m.FirstName, m.LastName, COUNT(l.LoanID) AS BooksLoaned
FROM Members m
LEFT JOIN Loans l ON m.MemberID = l.MemberID
GROUP BY m.MemberID;
-- Indexing strategy to improve query performance
CREATE INDEX idx_loans_bookid ON Loans(BookID);
CREATE INDEX idx_loans_memberid ON Loans(MemberID);
CREATE INDEX idx_loans_loan_date ON Loans(LoanDate);
CREATE INDEX idx_books_title_author ON Books(Title, Author);
CREATE INDEX idx_members_email ON Members(Email);
```

Output:

SQLite

SQL ▾ 1 / 1 1 - 2 of 2

Title	Author	FirstName	LastName	LoanDate
The Great Gatsby	F. Scott Fitzgerald	John	Doe	2024-05-01
To Kill a Mockingbird	Harper Lee	Jane	Smith	2024-05-15

SQL ▾ 1 / 1 1 - 2 of 2

Title	Author	FirstName	LastName	LoanDate
The Great Gatsby	F. Scott Fitzgerald	John	Doe	2024-05-01
To Kill a Mockingbird	Harper Lee	Jane	Smith	2024-05-15

SQL ▾ 1 / 1 1 - 2 of 2

FirstName	LastName	BooksLoaned
John	Doe	1
Jane	Smith	1

Justification:

The system is designed using three connected tables, where the Loans table links the Books and Members tables. This helps keep the data organized and avoids repeating the same information. SQL joins and conditions are used to combine and analyze the data, making it easy to track which books are issued, find overdue books, and calculate how many books each member has borrowed.

Task 4: Op mize Queries

Prompt:

Analyze and improve the performance of SQL queries using an example. Start by writing the original query that uses a subquery to find books written by authors from the UK, and then rewrite it using a more efficient method with joins. Make sure both queries give the same result. Briefly explain how the improved query is faster and works better for large data. Also suggest simple improvements like adding indexes on important columns (such as author_id and country) and avoiding unnecessary subqueries. Provide both the original and improved queries, along with a short step to check that the results are correct. Ensure the final output is clear, simple, and works properly in SQLite.

Code:

```
-- SQLite
-- Analyze and optimize SQL queries for better performance using a given example

PRAGMA foreign_keys = ON;

-- Create Authors table CREATE TABLE Authors (
id INTEGER PRIMARY KEY AUTOINCREMENT,
name TEXT NOT NULL,     country TEXT NOT
NULL
);

-- Create Books table CREATE TABLE Books (    book_id
INTEGER PRIMARY KEY AUTOINCREMENT,    title
TEXT NOT NULL,     author_id INTEGER,
   FOREIGN KEY (author_id) REFERENCES Authors(id)
);

-- Insert sample data
INSERT INTO Authors (name, country) VALUES
('Author A', 'UK'),
('Author B', 'USA'),
('Author C', 'UK');

INSERT INTO Books (title, author_id) VALUES
('Book 1', 1),
('Book 2', 2),
('Book 3', 3);

-- Original Query using subquery
SELECT title
FROM Books
WHERE author_id IN (
   SELECT id
   FROM Authors
   WHERE country = 'UK'
);

-- Optimized Query using JOIN
SELECT b.title
FROM Books b
JOIN Authors a ON b.author_id = a.id
WHERE a.country = 'UK';

-- Validation query (should return empty if both are same)
SELECT title
FROM Books
WHERE author_id IN (
   SELECT id FROM Authors WHERE country = 'UK' )
```

```
EXCEPT

SELECT b.title
FROM Books b
JOIN Authors a ON b.author_id = a.id
WHERE a.country = 'UK';

-- Indexing for optimization
CREATE INDEX idx_books_author_id ON Books(author_id); CREATE INDEX
idx_authors_country ON Authors(country);

-- Explanation:
-- The original query uses a subquery which may scan data multiple times.
-- The optimized query uses JOIN, allowing faster execution and better use of indexes.
```

Output:

Justification

The original query uses a subquery, which may be slower when working with large amounts of data, while the optimized query uses a join to make it run faster. Adding indexes and avoiding nested queries further improves performance, while still giving the same results.

Task 5: University Course Registration

Prompt:

Create a complete SQL solution for a University Course Registration System by designing a well-organized database with four tables: Students, Courses, Faculty, and Registrations. Define primary keys and foreign keys, use suitable data types, and apply constraints like NOT NULL and UNIQUE so that it works correctly in SQLite. Set up relationships so that each student can register for multiple courses and each course is taught by one faculty member, while following good design rules (up to Third Normal Form). Then, write SQL queries using joins to list all students enrolled in a specific course using course_id, find faculty members who teach more than

two courses, and get the courses with the highest number of registrations. Also include sample INSERT statements for testing, and ensure the final SQL code is clear, well-structured, and runs without any errors.

Code:

```sql
-- SQLite
--Generate a complete SQL solution for a University Course Registration System by creating a normalized database schema with four tables: Students, Courses, Faculty, and
Registrations. Include primary keys, foreign keys, appropriate data types, and constraints such as NOT NULL and UNIQUE, ensuring compatibility with SQLite. Define relationships such that each student can register for multiple courses and each course is taught by one faculty member. Maintain at least Third Normal Form (3NF). Then write SQL queries using joins to list all students enrolled in a specific course using course_id, find faculty members teaching more than 2 courses, and retrieve courses with the highest number of registrations. Include sample INSERT statements for testing and ensure the output is clean, well-structured, and fully executable without errors.
-- Create the Students table CREATE TABLE Students (
student_id INTEGER PRIMARY KEY AUTOINCREMENT,
first_name TEXT NOT NULL,     last_name TEXT NOT NULL,
email TEXT NOT NULL UNIQUE
);
-- Create the Faculty table CREATE TABLE Faculty (
faculty_id INTEGER PRIMARY KEY AUTOINCREMENT,
first_name TEXT NOT NULL,       last_name TEXT NOT
NULL,     email TEXT NOT NULL UNIQUE
);
-- Create the Courses table CREATE TABLE Courses (
course_id INTEGER PRIMARY KEY AUTOINCREMENT,
course_name TEXT NOT NULL,     faculty_id INTEGER NOT
NULL,
    FOREIGN KEY (faculty_id) REFERENCES Faculty(faculty_id) );
-- Create the Registrations table CREATE TABLE Registrations (
registration_id INTEGER PRIMARY KEY AUTOINCREMENT,
student_id INTEGER NOT NULL,     course_id INTEGER NOT NULL,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id),
    UNIQUE(student_id, course_id)
);
-- Sample INSERT statements for testing
INSERT INTO Faculty (first_name, last_name, email) VALUES ('John', 'Doe',
'john.doe@university.edu');
INSERT INTO Students (first_name, last_name, email) VALUES ('Alice', 'Smith',
'alice.smith@university.edu');
INSERT INTO Courses (course_name, faculty_id) VALUES ('Introduction to Computer Science', 1);
INSERT INTO Registrations (student_id, course_id) VALUES (1, 1);
```

```sql
-- Query to list all students enrolled in a specific course using course_id
SELECT s.first_name, s.last_name, s.email
FROM Students s
JOIN Registrations r ON s.student_id = r.student_id
WHERE r.course_id = 1;
-- Query to find faculty members teaching more than 2 courses
SELECT f.first_name, f.last_name, COUNT(c.course_id) AS course_count
FROM Faculty f
JOIN Courses c ON f.faculty_id = c.faculty_id
GROUP BY f.faculty_id
HAVING course_count > 2;
-- Query to retrieve courses with the highest number of registrations
SELECT c.course_name, COUNT(r.registration_id) AS registration_count FROM Courses c
JOIN Registrations r ON c.course_id = r.course_id
GROUP BY c.course_id
ORDER BY registration_count DESC
LIMIT 1;
```

Output:

SQL ▾ < 1 / 1 > 1 - 1 of 1

first_name	last_name	email
Alice	Smith	alice.smith@university.edu

SQL ▾ < 1 / 1 > 1 - 0 of 0

SQL ▾ < 1 / 1 > 1 - 1 of 1

course_name	registration_count
Introduction to Computer Science	1

Justification

The Registrations table connects the Students and Courses tables, allowing a student to enroll in multiple courses and handling the many-to-many relationship in an efficient way. This design helps keep the database well-organized and avoids duplicate data. SQL joins and aggregation functions are used to analyze enrollments, understand faculty workload, and identify popular courses. Proper use of primary and foreign keys ensures data integrity, while constraints like NOT NULL and UNIQUE improve data accuracy. Additionally, indexing important columns can improve query performance, making the system faster and more efficient when working with large amounts of data.